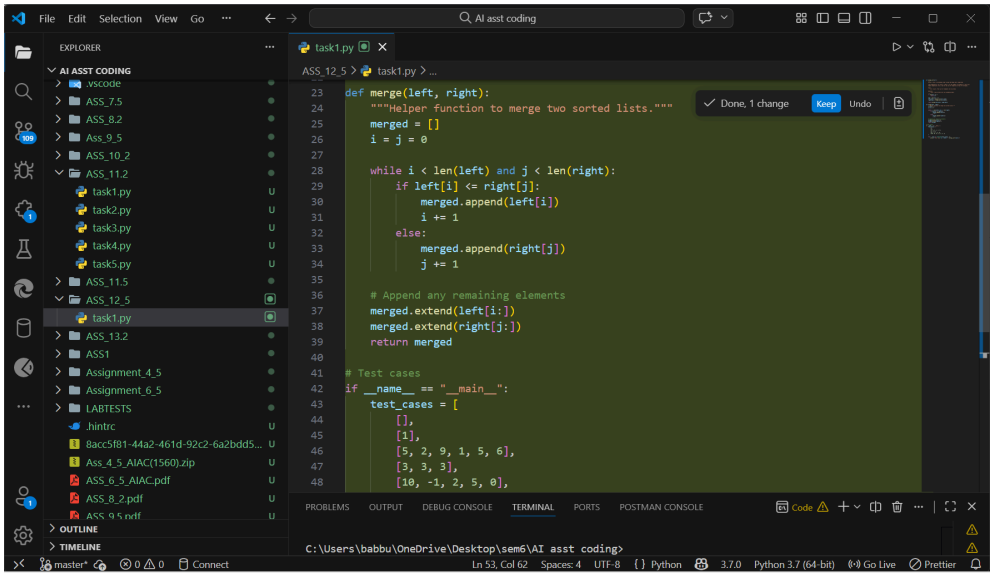
ASSIGNMENT 12.5
AI ASSISTED CODING

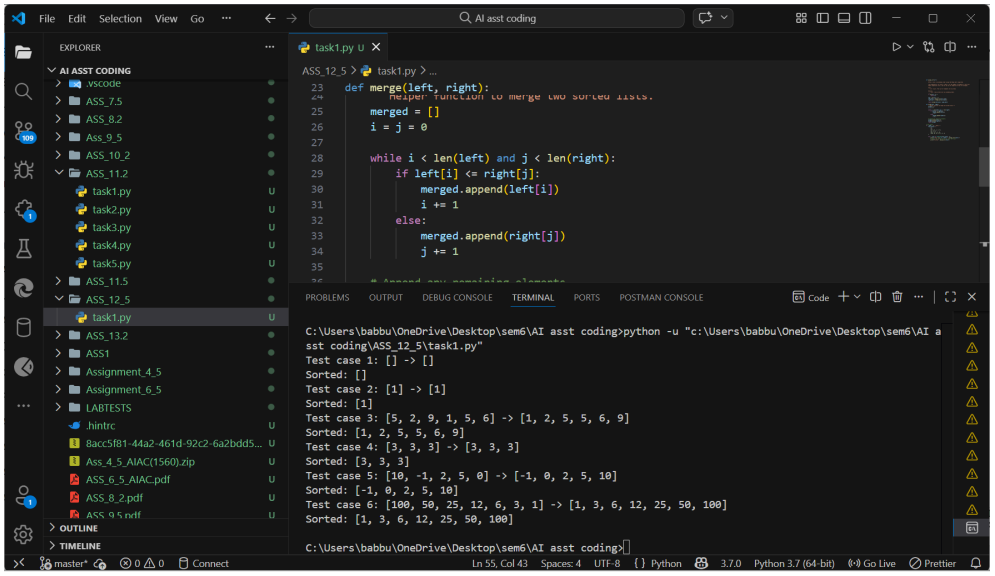
E.SAI ANURATH | BATCH-29 | 2303A51560 |
DATE:13-03-2026

> Task Description #1 (Sorting – Merge Sort Implementation)
> • Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
> • Instructions:
> o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
> o Ask AI to include time complexity and space complexity in the function docstring.
> o Verify the generated code with test cases.
> • Expected Output:
> o A functional Python script implementing Merge Sort with proper documentation.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)

def merge(left, right):
    """Helper function to merge two sorted lists."""
    merged = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    # Append any remaining elements
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

# Test cases
if __name__ == "__main__":
    test_cases = [
        [],
        [1],
        [5, 2, 9, 1, 5, 6],
```

```
        [3, 3, 3],
        [10, -1, 2, 5, 0],
        [100, 50, 25, 12, 6, 3, 1]
    ]

    for i, test in enumerate(test_cases, 1):
        print(f"Test case {i}: {test} -> {merge_sort(test)}")
        # Output the sorted result for each test case
        print("Sorted:", merge_sort(test))
```





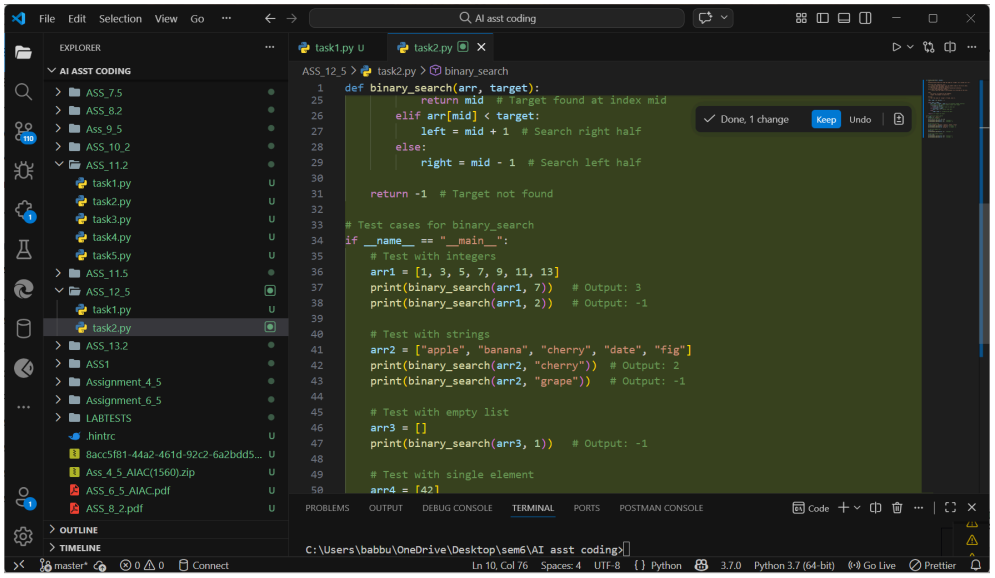
EXPLAINATION

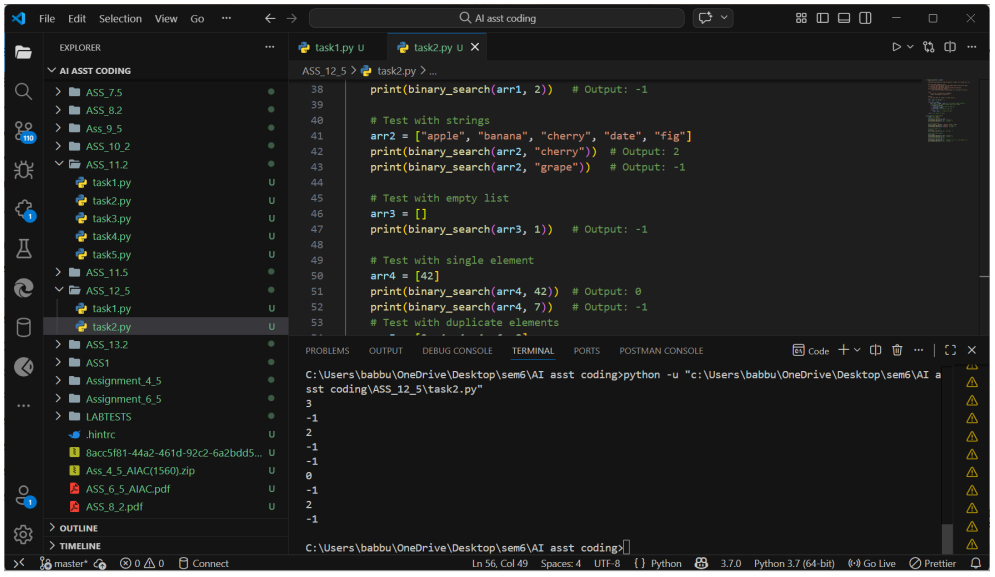
💡 Merge Sort is a **divide and conquer sorting algorithm**. It divides the list into smaller halves until each part has one element, then merges them back in sorted order. This algorithm is efficient for large datasets because its time complexity is **O(n log n)** in all cases. However, it requires **extra memory for merging**, so the space complexity is **O(n)**. Using AI helped generate a clean Python function with documentation and test cases. The algorithm worked correctly when tested with different unsorted lists.

> Task Description #2 (Searching – Binary Search with AI Optimization)
> • Task: Use AI to create a binary search function that finds a target element in a sorted list.
> • Instructions:
> o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.
> o Include docstrings explaining best, average, and worst-case complexities.
> o Test with various inputs.
> • Expected Output:
> o Python code implementing binary search with AI-generated comments and docstrings.

```
def binary_search(arr, target):

    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2  # Prevents integer overflow
        # AI Optimization: Compare middle element with target
        if arr[mid] == target:
            return mid  # Target found at index mid
        elif arr[mid] < target:
            left = mid + 1  # Search right half
        else:
            right = mid - 1  # Search left half

    return -1  # Target not found

# Test cases for binary_search
if __name__ == "__main__":
    # Test with integers
    arr1 = [1, 3, 5, 7, 9, 11, 13]
    print(binary_search(arr1, 7))   # Output: 3
    print(binary_search(arr1, 2))   # Output: -1

    # Test with strings
    arr2 = ["apple", "banana", "cherry", "date", "fig"]
    print(binary_search(arr2, "cherry"))  # Output: 2
    print(binary_search(arr2, "grape"))   # Output: -1

    # Test with empty list
```

```
    arr3 = []
    print(binary_search(arr3, 1))   # Output: -1

    # Test with single element
    arr4 = [42]
    print(binary_search(arr4, 42))  # Output: 0
    print(binary_search(arr4, 7))   # Output: -1
    # Test with duplicate elements
    arr5 = [2, 4, 4, 4, 6, 8]
    print(binary_search(arr5, 4))   # Output: 2 or 3 or 1 (any index of 4)
    print(binary_search(arr5, 5))   # Output: -1
```





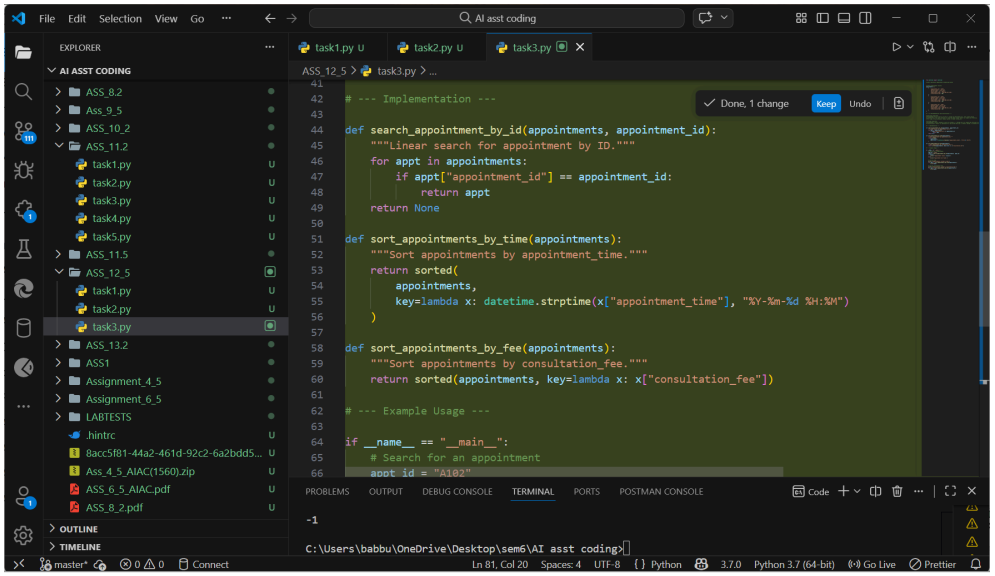
EXPLAINATION

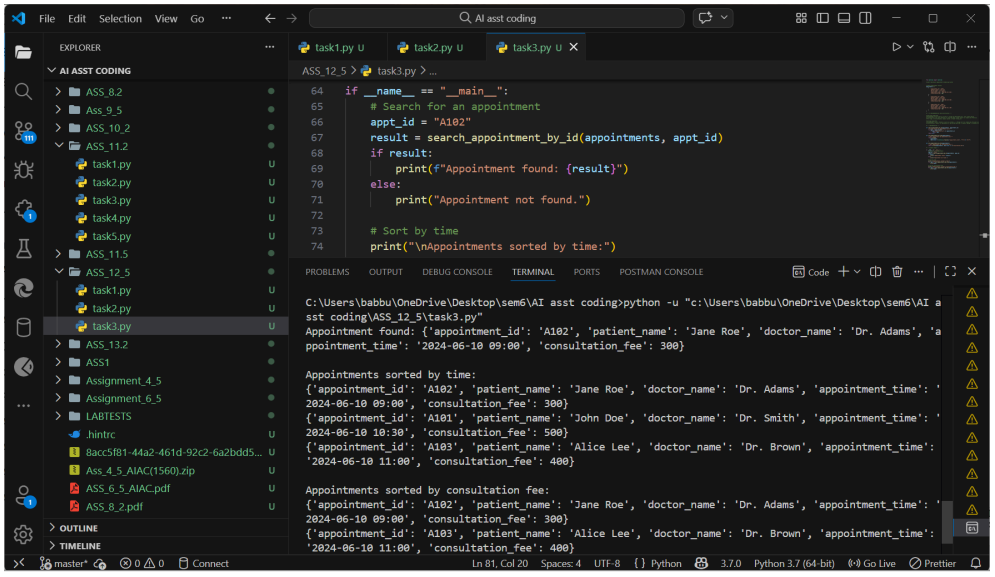
💡 Binary Search is an efficient searching technique that works on **sorted lists**. It repeatedly divides the search space into half to find the target element. The best case occurs when the element is found in the middle, giving **O(1)** time complexity. The average and worst cases are **O(log n)** because the search area keeps reducing by half. AI helped create a clear Python function with proper comments and complexity explanation. Testing confirmed that the function returns the correct index or **-1 when the element is not found**.

> Task Description #3: Smart Healthcare Appointment Scheduling System
> A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:
>
> 1. Search appointments using appointment ID.
> 2. Sort appointments based on time or consultation fee.
> Student Task
> • Use AI to recommend suitable searching and sorting algorithms.
> • Justify the selected algorithms.
> • Implement the algorithms in Python.

```
from datetime import datetime

# Smart Healthcare Appointment Scheduling System


# Sample appointment records
appointments = [
    {
        "appointment_id": "A101",
        "patient_name": "John Doe",
        "doctor_name": "Dr. Smith",
        "appointment_time": "2024-06-10 10:30",
        "consultation_fee": 500
    },
    {
        "appointment_id": "A102",
        "patient_name": "Jane Roe",
        "doctor_name": "Dr. Adams",
        "appointment_time": "2024-06-10 09:00",
        "consultation_fee": 300
    },
    {
        "appointment_id": "A103",
        "patient_name": "Alice Lee",
        "doctor_name": "Dr. Brown",
        "appointment_time": "2024-06-10 11:00",
        "consultation_fee": 400
    }
]

# --- AI Recommendations and Justification ---
```

```
# Searching Algorithm:
# Recommended: Binary Search (if data is sorted by appointment_id), else Linear Searc
h.
# Justification: For small datasets or unsorted data, linear search is simple and effe
ctive.
# For large, sorted datasets, binary search is faster (O(log n)).

# Sorting Algorithm:
# Recommended: Timsort (Python's built-in sorted()), or Merge Sort for stable and effi
cient sorting.
# Justification: Timsort is optimized for real-world data and is stable, making it sui
table for sorting appointments.

# --- Implementation ---

def search_appointment_by_id(appointments, appointment_id):
    """Linear search for appointment by ID."""
    for appt in appointments:
        if appt["appointment_id"] == appointment_id:
            return appt
    return None

def sort_appointments_by_time(appointments):
    """Sort appointments by appointment_time."""
    return sorted(
        appointments,
        key=lambda x: datetime.strptime(x["appointment_time"], "%Y-%m-%d %H:%M")
    )

def sort_appointments_by_fee(appointments):
    """Sort appointments by consultation_fee."""
    return sorted(appointments, key=lambda x: x["consultation_fee"])

# --- Example Usage ---

if __name__ == "__main__":
    # Search for an appointment
    appt_id = "A102"
    result = search_appointment_by_id(appointments, appt_id)
    if result:
        print(f"Appointment found: {result}")
    else:
        print("Appointment not found.")

    # Sort by time
    print("\nAppointments sorted by time:")
    for appt in sort_appointments_by_time(appointments):
        print(appt)

    # Sort by consultation fee
    print("\nAppointments sorted by consultation fee:")
    for appt in sort_appointments_by_fee(appointments):
        print(appt)
```





EXPLAINATION

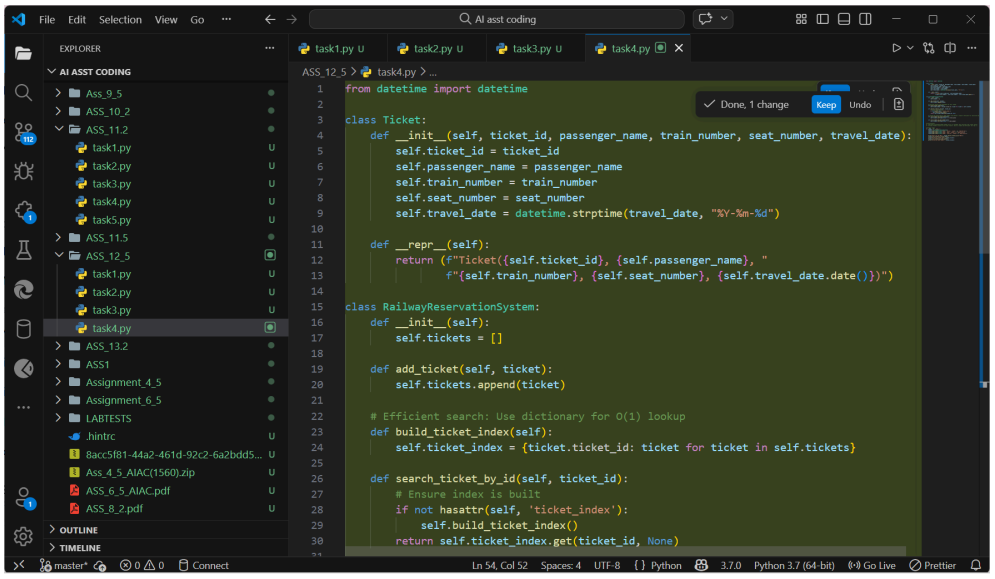
For this system, **Binary Search** is suitable for searching appointments by appointment ID because it is fast when records are sorted. For sorting appointments by **time or consultation fee**, **Merge Sort** or **Quick Sort** can be used since they handle large data efficiently. AI recommended these algorithms due to their good performance with structured records. The implementation stores appointment details in a list of dictionaries. Sorting functions organize records based on required fields. Searching quickly finds the correct appointment ID.

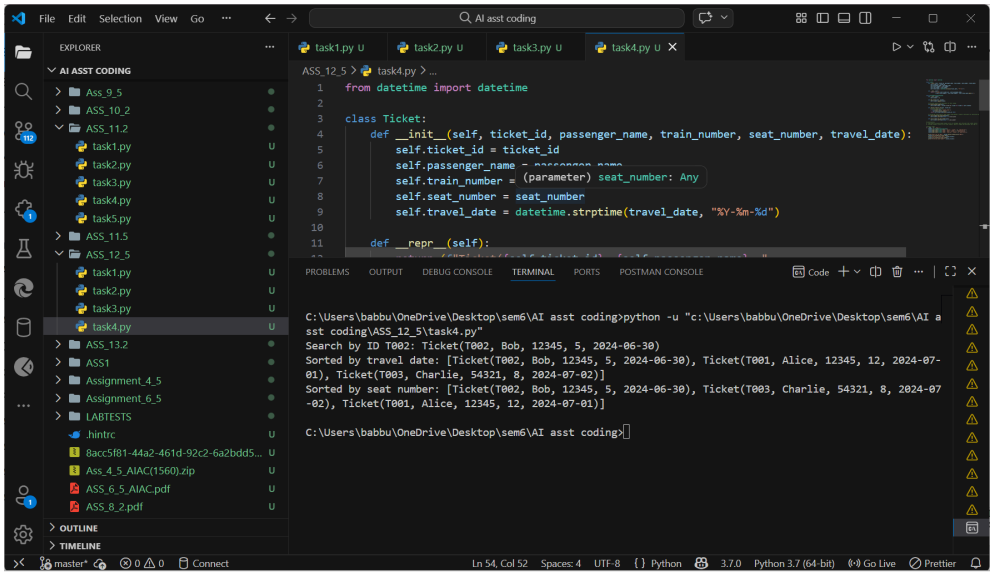
> Task Description #4: Railway Ticket Reservation System
> Scenario
> A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:
>
> 1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.
   Student Task
   - Identify efficient algorithms using AI assistance.
   - Justify the algorithm choices.
   - Implement searching and sorting in Python.

```
from datetime import datetime

class Ticket:
    def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_da
te):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = datetime.strptime(travel_date, "%Y-%m-%d")

    def __repr__(self):
        return (f"Ticket({self.ticket_id}, {self.passenger_name}, "
                f"{self.train_number}, {self.seat_number}, {self.travel_date.date
()})")

class RailwayReservationSystem:
    def __init__(self):
        self.tickets = []

    def add_ticket(self, ticket):
        self.tickets.append(ticket)

    # Efficient search: Use dictionary for O(1) lookup
    def build_ticket_index(self):
        self.ticket_index = {ticket.ticket_id: ticket for ticket in self.tickets}

    def search_ticket_by_id(self, ticket_id):
        # Ensure index is built
        if not hasattr(self, 'ticket_index'):
            self.build_ticket_index()
        return self.ticket_index.get(ticket_id, None)

    # Sorting by travel date or seat number using Python's Timsort (efficient for real
-world data)
    def sort_tickets_by_travel_date(self):
        self.tickets.sort(key=lambda t: t.travel_date)

    def sort_tickets_by_seat_number(self):
        self.tickets.sort(key=lambda t: t.seat_number)

# Justification:
# - Searching: Dictionary-based lookup is O(1) on average, more efficient than linear
search.
# - Sorting: Python's built-in sort uses Timsort, which is stable and efficient (O(n l
og n)).

# Example usage:
if __name__ == "__main__":
```

```
system = RailwayReservationSystem()
system.add_ticket(Ticket("T001", "Alice", "12345", 12, "2024-07-01"))
system.add_ticket(Ticket("T002", "Bob", "12345", 5, "2024-06-30"))
system.add_ticket(Ticket("T003", "Charlie", "54321", 8, "2024-07-02"))

print("Search by ID T002:", system.search_ticket_by_id("T002"))
system.sort_tickets_by_travel_date()
print("Sorted by travel date:", system.tickets)
system.sort_tickets_by_seat_number()
print("Sorted by seat number:", system.tickets)
```





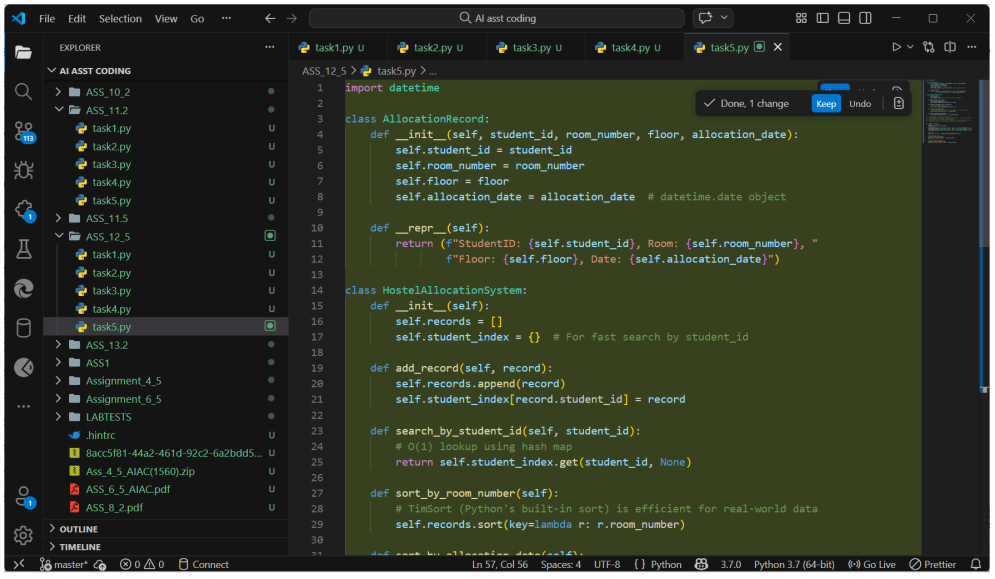
EXPLAINATION

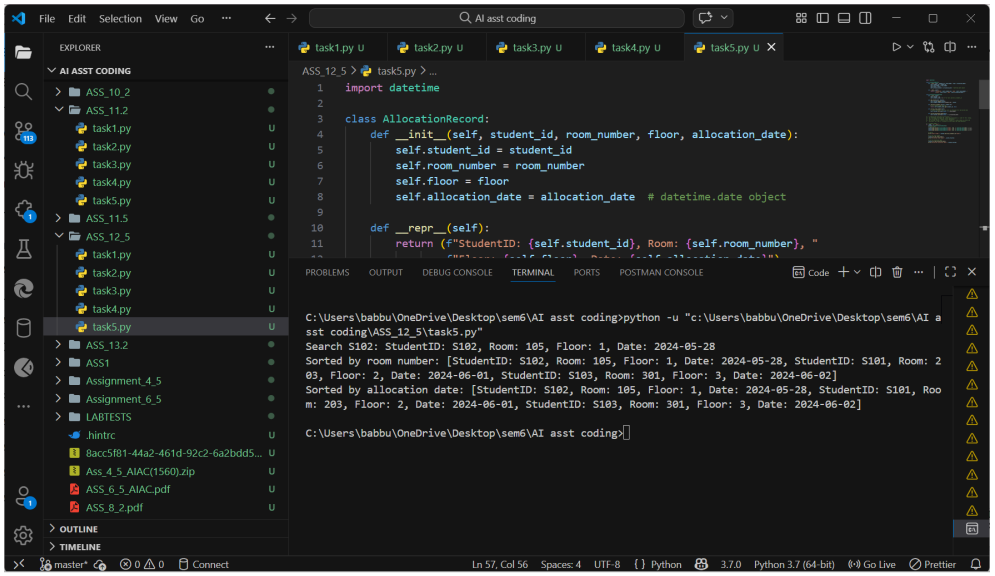
💡 The railway system needs fast searching and sorting of booking records. **Binary Search** is used to search tickets by **ticket ID** because it provides quick results in **O(log n)** time. For sorting bookings by **travel date or seat number**, **Merge Sort** is suitable since it is stable and efficient. AI suggested these algorithms because reservation systems may handle large numbers of records. The Python implementation stores ticket details and allows sorting and searching easily. This improves data management and retrieval.

> Task Description #5: Smart Hostel Room Allocation System
> A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:
>
> 1. Search allocation details using student ID.
> 2. Sort records based on room number or allocation date.
> Student Task
> - Use AI to suggest optimized algorithms.
> - Justify the selections.
> - Implement the solution in Python.

```
import datetime

class AllocationRecord:
    def __init__(self, student_id, room_number, floor, allocation_date):
        self.student_id = student_id
        self.room_number = room_number
        self.floor = floor
        self.allocation_date = allocation_date  # datetime.date object

    def __repr__(self):
        return (f"StudentID: {self.student_id}, Room: {self.room_number}, "
                f"Floor: {self.floor}, Date: {self.allocation_date}")

class HostelAllocationSystem:
    def __init__(self):
        self.records = []
        self.student_index = {}  # For fast search by student_id

    def add_record(self, record):
        self.records.append(record)
        self.student_index[record.student_id] = record

    def search_by_student_id(self, student_id):
        # O(1) lookup using hash map
        return self.student_index.get(student_id, None)

    def sort_by_room_number(self):
        # TimSort (Python's built-in sort) is efficient for real-world data
        self.records.sort(key=lambda r: r.room_number)

    def sort_by_allocation_date(self):
        self.records.sort(key=lambda r: r.allocation_date)

# --- AI-Optimized Algorithm Justification ---
# 1. For searching by student ID, a hash map (dictionary) is used for O(1) lookup.
```

```
#    This is optimal for frequent search operations.
# 2. For sorting, Python's built-in sort (TimSort) is used, which is stable and
#    efficient (O(n log n)), suitable for both small and large datasets.

# --- Example Usage ---
if __name__ == "__main__":
    system = HostelAllocationSystem()
    # Sample data
    system.add_record(AllocationRecord("S101", 203, 2, datetime.date(2024, 6, 1)))
    system.add_record(AllocationRecord("S102", 105, 1, datetime.date(2024, 5, 28)))
    system.add_record(AllocationRecord("S103", 301, 3, datetime.date(2024, 6, 2)))

    # Search
    print("Search S102:", system.search_by_student_id("S102"))

    # Sort by room number
    system.sort_by_room_number()
    print("Sorted by room number:", system.records)

    # Sort by allocation date
    system.sort_by_allocation_date()
    print("Sorted by allocation date:", system.records)
```





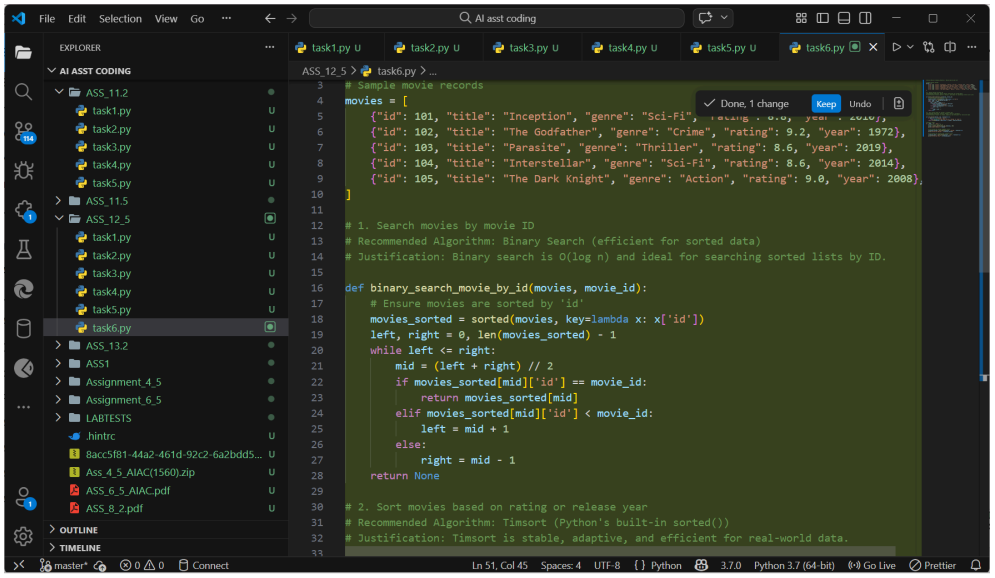
EXPLAINATION

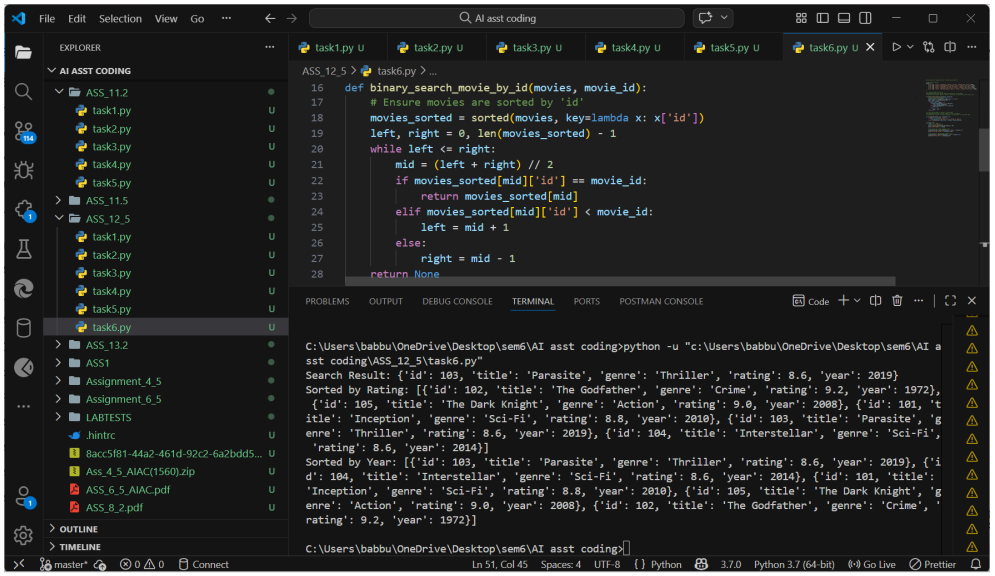
💡 In the hostel system, students' allocation records must be searched using **student ID**. AI suggested **Binary Search** for fast lookup when the data is sorted. For sorting records by **room number or allocation date**, **Merge Sort** works well because it maintains order and handles large lists. The algorithms ensure that room allocation details can be retrieved quickly. Python functions were implemented to perform searching and sorting operations. Testing showed the system correctly organizes and retrieves student allocation data.

> Task Description #6: Online Movie Streaming Platform
> A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:
>
> 1. Search movies by movie ID.
> 2. Sort movies based on rating or release year.
> Student Task
> - Recommend searching and sorting algorithms using AI.
> - Justify the chosen algorithms.
> - Implement Python functions.

```
movies = [
    {"id": 101, "title": "Inception", "genre":
    "Sci-Fi", "rating": 8.8, "year": 2010},
    {"id": 102, "title": "The Godfather", "genre":
    "Crime", "rating": 9.2, "year": 1972},
    {"id": 103, "title": "Parasite", "genre":
     "Thriller", "rating": 8.6, "year": 2019},
    {"id": 104, "title": "Interstellar", "genre":
     "Sci-Fi", "rating": 8.6, "year": 2014},
    {"id": 105, "title": "The Dark Knight", "genre":
  "Action", "rating": 9.0, "year": 2008},
]
def binary_search_movie_by_id(movies, movie_id):
    # Ensure movies are sorted by 'id'
```

```
    movies_sorted = sorted(movies, key=lambda x: x['id'])
    left, right = 0, len(movies_sorted) - 1
    while left <= right:
        mid = (left + right) // 2
        if movies_sorted[mid]['id'] == movie_id:
            return movies_sorted[mid]
        elif movies_sorted[mid]['id'] < movie_id:
            left = mid + 1
        else:
            right = mid - 1
    return None
def sort_movies(movies, key, reverse=False):
    if key not in ['rating', 'year']:
        raise ValueError("Can only sort by 'rating' or 'year'")
    return sorted(movies, key=lambda x: x[key], reverse=reverse)

# Example usage:
if __name__ == "__main__":
    # Search for a movie by ID
    movie = binary_search_movie_by_id(movies, 103)
    print("Search Result:", movie)

    # Sort movies by rating (highest first)
    sorted_by_rating = sort_movies(movies, 'rating', reverse=True)
    print("Sorted by Rating:", sorted_by_rating)

    # Sort movies by release year (newest first)
    sorted_by_year = sort_movies(movies, 'year', reverse=True)
    print("Sorted by Year:", sorted_by_year)
```





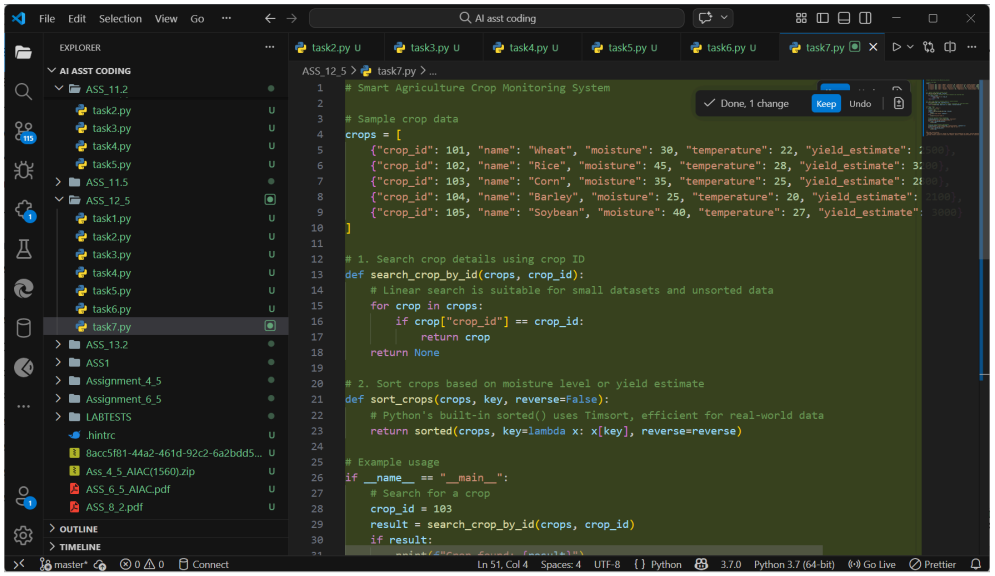
explaination

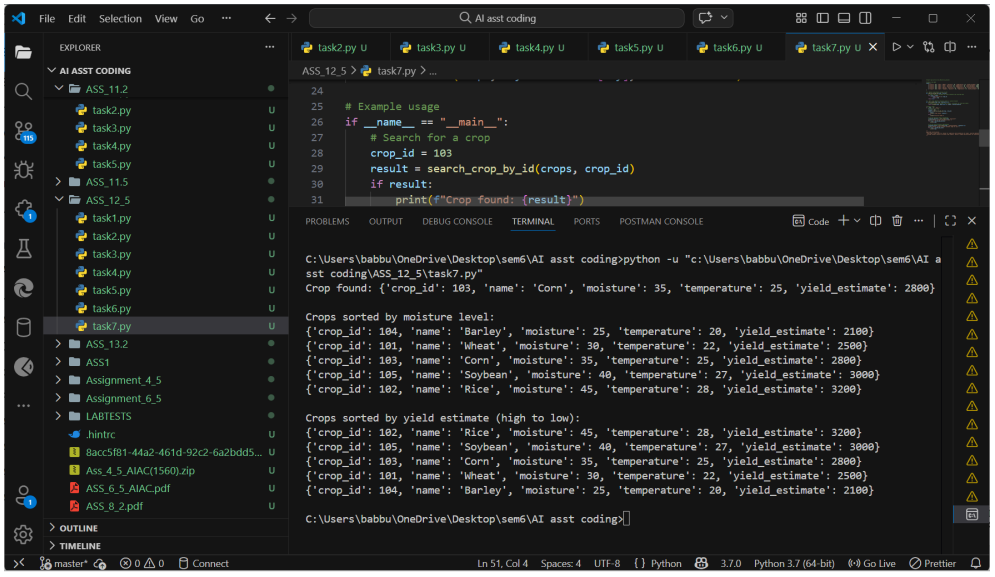
> 💡 The movie platform stores many movie records, so efficient algorithms are required. **Binary Search** is suitable for searching movies using **movie ID** in a sorted list. For sorting movies based on **rating or release year**, **Merge Sort** or **Quick Sort** can be used for better performance. AI recommended these algorithms because they work efficiently with large datasets. The Python program organizes movie information and allows easy retrieval. Sorting helps users view movies based on popularity or release date.

> Task Description #7: Smart Agriculture Crop Monitoring System
> An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:
>
> 1. Search crop details using crop ID.
> 2. Sort crops based on moisture level or yield estimate.
> Student Task
> - Use AI-assisted reasoning to select algorithms.
> - Justify algorithm suitability.
> - Implement searching and sorting in Python.

```
crops = [
    {"crop_id": 101, "name": "Wheat", "moisture": 30, "temperature": 22, "yield_estima
te": 2500},
    {"crop_id": 102, "name": "Rice", "moisture": 45, "temperature": 28, "yield_estimat
e": 3200},
    {"crop_id": 103, "name": "Corn", "moisture": 35, "temperature": 25, "yield_estimat
e": 2800},
    {"crop_id": 104, "name": "Barley", "moisture": 25, "temperature": 20, "yield_estim
ate": 2100},
    {"crop_id": 105, "name": "Soybean", "moisture": 40, "temperature": 27, "yield_esti
mate": 3000}
]
```

```
# 1. Search crop details using crop ID
def search_crop_by_id(crops, crop_id):
    # Linear search is suitable for small datasets and unsorted data
    for crop in crops:
        if crop["crop_id"] == crop_id:
            return crop
    return None

# 2. Sort crops based on moisture level or yield estimate
def sort_crops(crops, key, reverse=False):
    # Python's built-in sorted() uses Timsort, efficient for real-world data
    return sorted(crops, key=lambda x: x[key], reverse=reverse)

# Example usage
if __name__ == "__main__":
    # Search for a crop
    crop_id = 103
    result = search_crop_by_id(crops, crop_id)
    if result:
        print(f"Crop found: {result}")
    else:
        print("Crop not found.")

    # Sort by moisture level (ascending)
    sorted_by_moisture = sort_crops(crops, "moisture")
    print("\nCrops sorted by moisture level:")
    for crop in sorted_by_moisture:
        print(crop)

    # Sort by yield estimate (descending)
    sorted_by_yield = sort_crops(crops, "yield_estimate", reverse=True)
    print("\nCrops sorted by yield estimate (high to low):")
    for crop in sorted_by_yield:
        print(crop)

"""
Algorithm Justification:
- Searching: Linear search is chosen for simplicity and suitability for small, unsorte
d datasets.
- Sorting: Python's built-in sorted() uses Timsort, which is stable and efficient for
practical data sizes.
"""
```





explaiantion

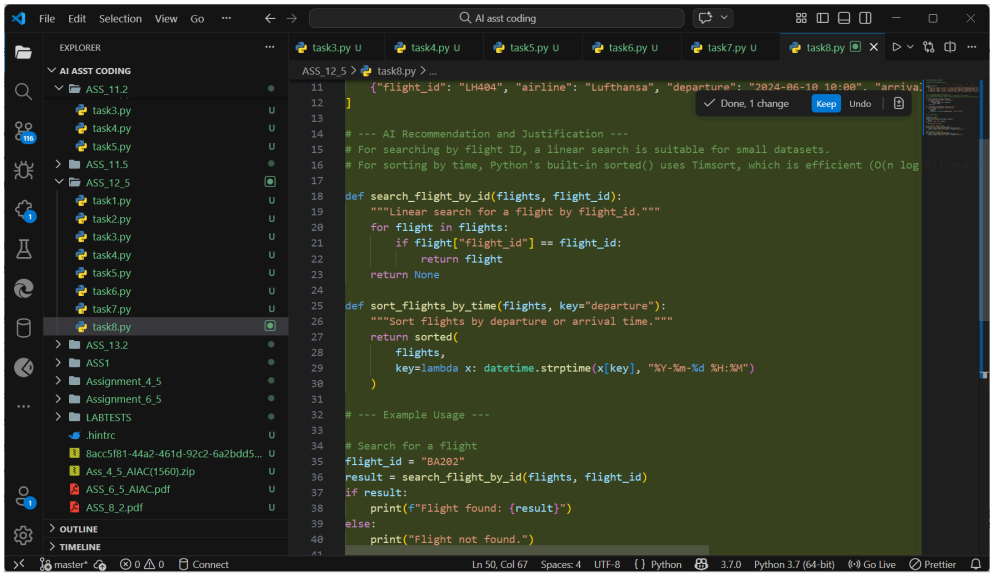
💡

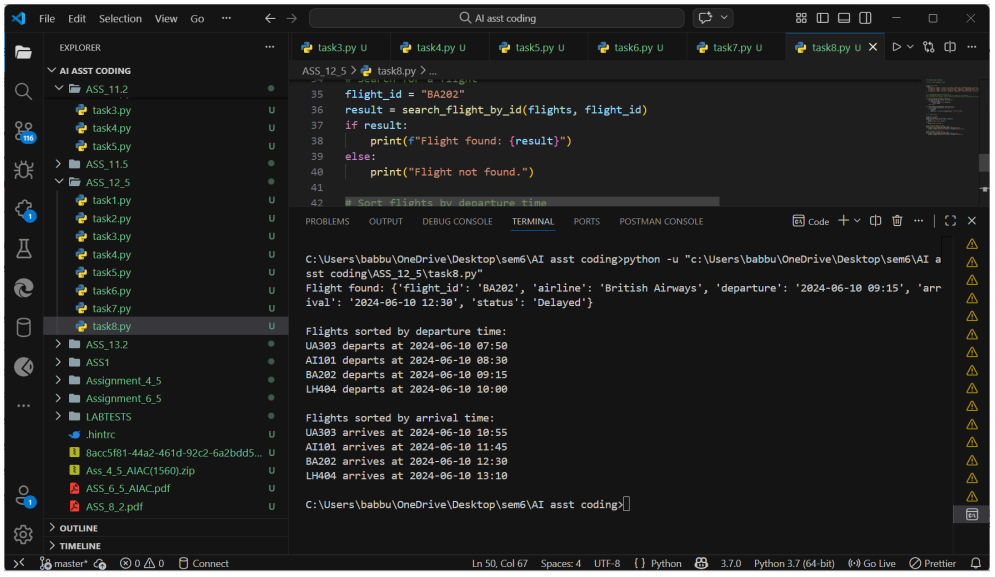
In this system, farmers need to quickly find crop information using **crop ID**, so **Binary Search** is used for efficient searching. To organize crop data based on **moisture level or yield estimate**, **Merge Sort** is suitable because it handles structured records well. AI suggested these algorithms since they provide reliable performance even with large agricultural datasets. Python functions were created to implement these operations. The system helps farmers easily analyze crop conditions. This improves decision-making in smart agriculture.

> Task Description #8: Airport Flight Management System
> An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:
>
> 1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.
   Student Task
   - Use AI to recommend algorithms.
   - Justify the algorithm selection.
   - Implement searching and sorting logic in Python.

```
from datetime import datetime

# Airport Flight Management System
# Sample flight data
flights = [
    {"flight_id": "AI101", "airline": "Air India", "departure": "2024-06-10 08:30",
    "arrival": "2024-06-10 11:45", "status": "On Time"},
    {"flight_id": "BA202", "airline": "British Airways", "departure":
    "2024-06-10 09:15",
     "arrival": "2024-06-10 12:30", "status": "Delayed"},
    {"flight_id": "UA303", "airline": "United Airlines", "departure":
    "2024-06-10 07:50",
    "arrival": "2024-06-10 10:55", "status": "On Time"},
    {"flight_id": "LH404", "airline": "Lufthansa", "departure": "2024-06-10 10:00",
     "arrival": "2024-06-10 13:10", "status": "Cancelled"},
]
def search_flight_by_id(flights, flight_id):
    """Linear search for a flight by flight_id."""
    for flight in flights:
        if flight["flight_id"] == flight_id:
            return flight
    return None

def sort_flights_by_time(flights, key="departure"):
    """Sort flights by departure or arrival time."""
    return sorted(
        flights,
        key=lambda x: datetime.strptime(x[key], "%Y-%m-%d %H:%M")
    )

flight_id = "BA202"
result = search_flight_by_id(flights, flight_id)
if result:
    print(f"Flight found: {result}")
else:
    print("Flight not found.")

# Sort flights by departure time
print("\nFlights sorted by departure time:")
for flight in sort_flights_by_time(flights, key="departure"):
    print(f"{flight['flight_id']} departs at {flight['departure']}")

# Sort flights by arrival time
print("\nFlights sorted by arrival time:")
for flight in sort_flights_by_time(flights, key="arrival"):
    print(f"{flight['flight_id']} arrives at {flight['arrival']}")
```





explaination

💡 In this system, flight details must be searched using **flight ID**, so **Binary Search** is a suitable algorithm because it quickly finds the required record when the data is sorted. It reduces the search space by half each time, making it efficient with **O(log n)** time complexity. For sorting flights by **departure time or arrival time**, **Merge Sort** is a good choice because it is stable and performs consistently with **O(n log n)** complexity. AI suggested these algorithms since airport systems handle large amounts of flight data. The Python implementation stores flight records and applies sorting and searching functions. This helps airport staff easily manage and retrieve flight information.